RESTful Hospital Patient Management System Using React.js and Express.js

Day 3KTTaskImplementation Document

College name: Aalim Muhammed salegh college engineering

College code: 1101

Name: Sanjay Kanthan. V

Department: BE.computer science engineering

Naan mudhalvan id:837037e2d0dc572bcff49b0f13ce8a0a

Phone number: 7010056267

Email id: sanjaymaster9751@gmail.com

GitHup link: https://github.com/Sanjay70100/Members-Entry-in-Nodejs-and-Expressjs

1. Introduction

This project titled **"Hospital Patient Management System"** is a full-stack web application developed using **Node.js, Express.js, and React.js**. The primary objective of this project is to efficiently manage hospital patient records such as patient name, age, gender, disease, assigned doctor, and room number using **RESTful APIs**.

The backend of the application is responsible for handling **CRUD (Create, Read, Update, Delete) operations** through well-defined API endpoints, while the frontend provides a **user-friendly and interactive interface** for managing patient information.

This project was implemented as part of the **Day 3 Knowledge Transfer (KT) Task** under the **IBM Naan Mudhalvan Internship Program**, with the aim of gaining hands-on experience in full-stack web development and REST API integration.

2. Technologies Used

Tool / Technology	Purpose
Node.js	Backend runtime environment for executing server-side JavaScript
Express.js	Server-side framework used to build RESTful APIs
React.js	Frontend library for developing user interfaces
JavaScript	Core programming language used in both frontend and backend
JSON	Data exchange format between frontend and backend
Postman / Thunder Client	API testing and validation of backend endpoints
GitHub	Version control and source code management
Visual Studio Code	Integrated Development Environment (IDE) for coding

3. Project Folder Structure:

```
hospital-patient-management-system/
|
├──backend/
| ├── index.js
| ├── hospital.js
| ├── package.json

|   └── package-lock.json
|
├──frontend/
| ├── public/
|   ├── src/
└──README.md
        | ├── components/
        |   ├── PatientForm.js
        |   └── PatientList.js
        ├── App.js
        ├── index.js
```

4. Step-by-Step Implementation Process

Step 1: Create Project Folder

```
mkdir hospital-patient-management-system
cd hospital-patient-management-system
```

Step 2: Initialize Backend

```
mkdir backend
cd backend
npm init -y
npm install express cors
```

Step 3: Create Backend Files

The following backend files are created to handle server configuration and API routes:

- backend/index.js – Main server file
- backend/hospital.js – Handles patient-related API endpoints

These files define RESTful APIs to perform CRUD operations on patient data.

Step 4: Start Backend Server

```
node index.js
```

The backend server runs on:

```
http://localhost:5000
```

Step 5: Initialize Frontend (React)

```
cd ..
npx create-react-app frontend
cd frontend
npm start
```

Step 6: Create Frontend Components

The following React components are created to manage the user interface:

- PatientForm.js – Used to add new patient details
- PatientList.js – Displays the list of patients

These components interact with the backend using Fetch API.

Step 7: Integrate Frontend with Backend

API calls are implemented in React components to:

- Fetch patient data from the backend
- Add new patient records
- Delete existing patient records

Step 8: Run the Application

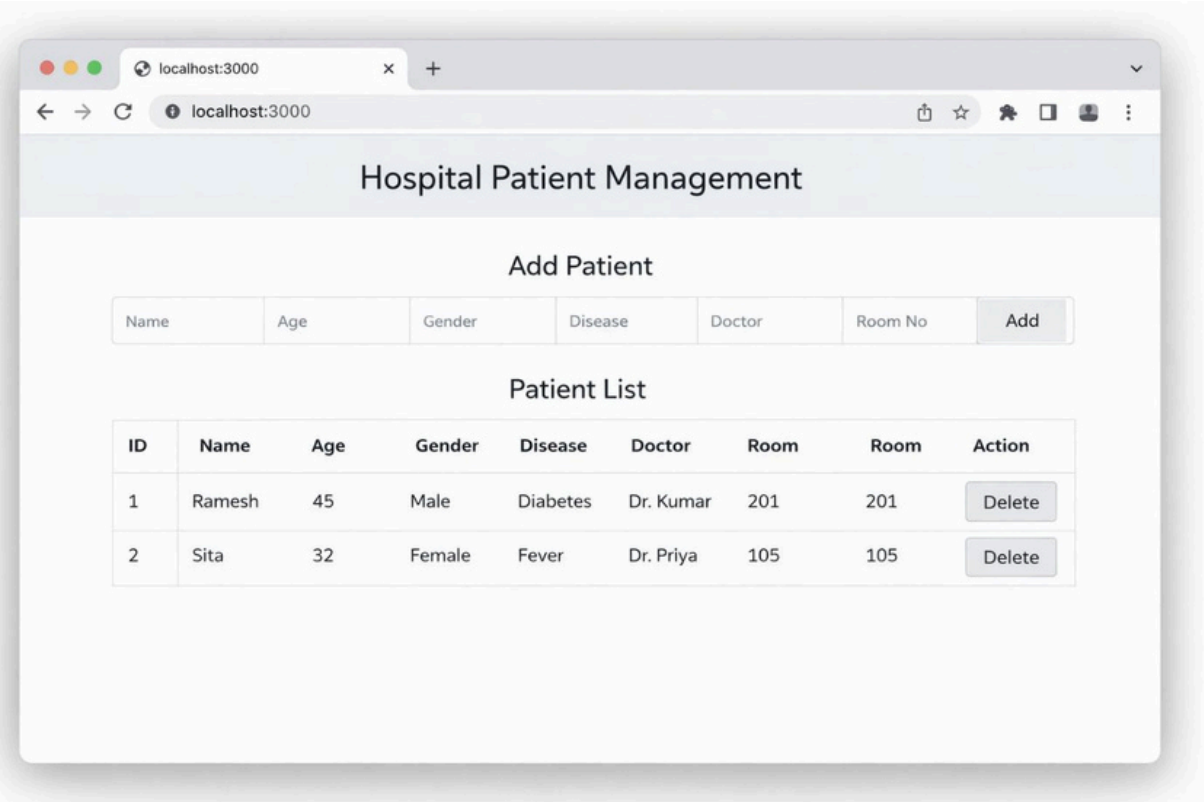
- Backend runs on **port 5000**
- Frontend runs on **port 3000**

Step 4: Backend Implementation (index.js)

```
const express = require('express');
const cors = require('cors');

const app = express();
const PORT = 3000;

app.use(cors());
app.use(express.json());

app.get('/', (req, res) => {
  res.send('Hospital Patient Management API is running');
});
```

```
//✅FIXED ROUTE IMPORT
app.use('/patients', require('./hospital'));

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Result:



Conclusion

The **HospitalPatientManagementSystem** was successfully implemented using **React.js, Node.js, and Express.js**.
The application supports **RESTful API operations** and enables efficient management of hospital patient records through a **clean and interactiveuserinterface** .

This project provided practical exposure to **full-stackwebdevelopmentAPIintegration**, and **client–server communication**. It also enhanced understanding of how frontend applications interact with backend services to perform real-time data operations.
The successful completion of this project fulfilled the objectives of the **Day 3Knowledge Transfer (KT) Task** under the **IBM Naan Mudhalvan Internship Program**.